

A Methodological Framework for Model-Based Self-management of Services and Components in Dependable Cyber-Physical Systems

DeJiu Chen^{1(✉)} and Zhonghai Lu²

¹ Mechatronics, Machine Design, School of ITM,
KTH Royal Institute of Technology, 100 44 Stockholm, Sweden
chendj@kth.se

² Electronics and Embedded Systems, School of ICT,
KTH Royal Institute of Technology, 164 40 Kista, Sweden
zhonghai@kth.se

Abstract. Modern automotive vehicles featuring ADAS (Advanced Driving Assistant Systems) and AD (Autonomous Driving) represent one category of dependable CPS (Cyber-Physical Systems). For such systems, the adaptation of generic purpose COTS (Commercial-Off-The-Shelf) services and components has been advocated in the industry as a necessary means for shortening the innovation loops and enabling efficient product evolution. This will however not be a trivial task due to the system safety- and time-criticality. This calls on one hand for formal specification of systems, and on the other hand for a systematic approach to module design, supervision and adaptations. Accordingly, we propose in this paper a novel method that emphasizes an integration of system models, formal contracts, and embedded services for effective self-management of COTS. The key modeling technologies include the EAST-ADL for formal system description and the A-G contract theory for module specification.

Keywords: Cyber-Physical systems · Commercial-Off-The-Shelf · Dependability · Real-time · EAST-ADL · Contract · Composition

1 Introduction

Embedded system in modern automotive vehicle allows more effective realization of advanced functionalities in a way that is impossible with pure mechanical and electrical solutions. It makes an automotive vehicle CP (Cyber-Physical) in nature by having both physical dynamics and energy flows under control, and the corresponding perception, control and cognitive loops with software and hardware solutions. Currently, the industry shares the view that the embedded system constitutes the most important technology for the advances in sustainability, road safety, and novel traffic solutions [1, 2]. For shortening the innovation loops and enabling efficient product evolution, the adaptation of generic purpose COTS (Commercial-Off-The-Shelf) solutions for the embedded system has been considered as necessary. Such COTS solutions typically

range from camera, radar and other sensors for traffic perception, to speech recognition and augmented-reality displays, and to wireless and telecommunication services for V2V and V2I connectivity. There are however often big gaps in regard to the conformity of qualities. For example, the expected lifetime for electronics components would be up to 15 years for automotive vehicles, in comparison to a length in 2–5 years for consumer products [3]. Meanwhile, the operational temperature ranges are normally $-40 \sim 160^{\circ}\text{C}$ for automotive vehicles and $0 \sim 40^{\circ}\text{C}$ for consumer products. Moreover, for the functional safety of automotive embedded systems, a systematic management of the risks of COTS according to the safety standard ISO26262 becomes necessary [4]. The information of concern typically includes not only a specification of its functional behaviors, but also a definition of its assumed failure modes, adopted quality assurance measures, and related mechanisms for fault tolerance and treatment. This is not a trivial task for a separately developed service or component.

This paper presents a novel approach that aims to act as a methodological framework for effective integration and management of separately developed services and components in safety critical CPS. The aim is to promote in general qualified CPS development and evolution across the boundaries of domains and organizations. The rest of this chapter is structured into the following sections: Sect. 2 provides an overview of the proposed methodological framework. Section 3 introduces EAST-ADL, which is the state-of-the-art technology adopted for systems modeling and modularity design. Section 4 presents the contract theory used as the formalism for module specification. Section 5 describes the corresponding run-time services developed for situation awareness and system adaptation. An overview of related technologies is given in Sect. 6. The paper concludes with Sect. 7.

2 The Methodological Framework

The methodology framework emphasizes an integration of the models for system design and the service for post-deployment quality management. It follows the knowledge-in-the-loop paradigm proposed in [5]. Shown in Fig. 1, the framework consists of the following four main steps:

- **Step I – System architecture design**, responsible for the *overall system design* in regard to the definition of system operational environments, the functional and technical constituent units, the expected behaviors and quality constraints, the corresponding verification and validation cases, etc. As the outcome, the design also stipulates the design-space with compositional variants and restrictions.
- **Step II – Partitioning and assertion specification**, responsible for *system modularity design*, which is centric on partitioning the target system into modules and thereby refining the system design for potential external COTS solutions. In effect, the design results in a restructuring of the target system so that some of its constituent units can be developed and managed independently. We specify the constraints with the *module assertions* with the allowed input and output signals, their preferred types and resolutions, execution time and criticalities, etc.

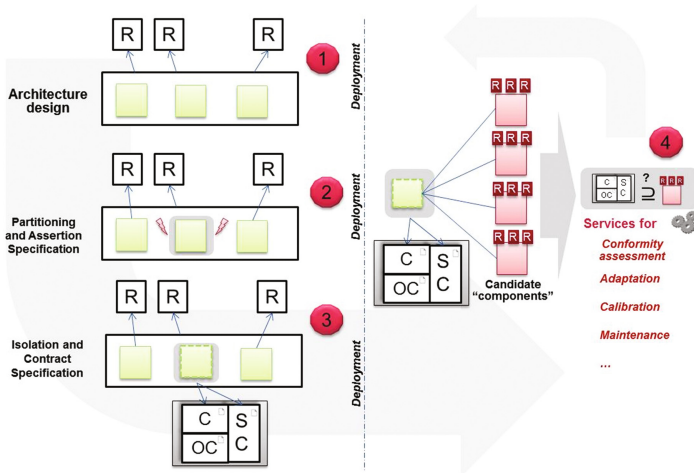


Fig. 1. The overall methodology for a model-based contract and service for self-management. ① Step I – Architecture design; ② Step II – Partitioning and Assertion Specification; ③ Step III – Isolation and Contract Specification; ④ Step IV – Service for self-management. R- Requirement; C- Contract; OC-Operation Contract; SC-Safety Contract.

- **Step III – Isolation and Contract Specification**, responsible for *module interface design* for the given modules. By *isolation*, we refer to the process of configuring the target system to allow each module, as a self-contained system unit, to be isolated from changes or variations in the target system and in the COTS. The results for modules are specified as *contracts*. Each contract formally specifies some assertions for successful COTS integration, on the functional, operational, and safety properties, referred to as Contract (C), Operation Contract (OC) and Safety Contract (SC) respectively in Fig. 1. While the functional properties focus on the input and output signal as well as the functions, the operational properties are related to the nominal operation and the modes for diagnostics and maintenance. The safety properties cover the failure modes, safe states, and the related transitions for error handling and fault treatment.
- **Step IV – Service-based Self-Management of System Services and Components**, responsible for the development and usage of *quality management services* for post-deployment monitoring, assessment and adaptation. These services take the module contracts as formal specifications of the parameters to be monitored, the conditions to be judged, and the adaptations to be conducted.

The realization of this methodology is based on the combination of some state-of-the-art technologies for system modeling, component specification, and self-management. We introduce these technologies by the follow-up sections.

3 EAST-ADL for System and Modularity Design

ADL (Architecture description language) is a modeling technology for structuring and managing the information of a system, relating to the *Step I* and *Step II* introduced previously. The EAST-ADL modeling framework, adopted here, represents a key European initiative towards a standardized description of automotive E/E systems [6, 7]. The EAST-ADL system model, together with the associated requirements and constraints, constitutes a basis for effective but still very flexible modularity design. That is, given the models capturing the system wide interdependencies, a particular modularization task can be driven at any specific abstraction level in accordance with the particular preferences. For example, the modules given by features (Vehicle Level) provide a structuring of the externally visible functionalities of the target system and allow a service-oriented composition of external functions. Meanwhile, module definitions can also be done for software components (Implementation Level) to allow a component-based engineering of software solutions. In both cases, the EAST-ADL system model constitutes a useful means for systematically reasoning about the corresponding module coupling and cohesion, both in regard to the functional interactions (which are given by communication links in the same level of abstraction) and technical implications (which are given by realization links across the abstraction levels). By capturing the associated requirements and constraints of system constituent units, an EAST-ADL model also provides a systematic modeling support for deriving the related module requirements and constraints.

4 A-G Contract for Formal Specification of Modules

Following the contract theory defined in [8, 9], we use formal contracts to specify the expected functional and technical properties of a system module, relating to the *Step III*. Fundamentally, a system module M is defined by its variables, behaviors as well as some related quality constraints as follows

$$M = (\mathbb{V}, \mathbb{F}, \mathbb{Q}) \quad (1)$$

with $\mathbb{V}$ for all variables $\mathbb{V} = \mathcal{U} \cup \mathcal{X} \cup \mathcal{Y} \cup \mathcal{K}$, where $\mathcal{U}$ denotes the input variables, $\mathcal{X}$ the internal variables, $\mathcal{Y}$ the output variables, and $\mathcal{K}$ the configuration variables; $\mathbb{F}$ for all functional behaviors over the variables $\mathbb{F} = \llbracket \mathcal{F}(\mathcal{U}, \mathcal{Y}, \mathcal{X}, \mathcal{K}) = 0 \rrbracket$; and $\mathbb{Q}$ for all quality constraints over the functional behaviors $\mathbb{Q} = \{\mathbb{P}, \mathbb{R}, \dots\} = \{\llbracket \mathcal{P}(\mathbb{F}) = 0 \rrbracket, \llbracket \mathcal{R}(\mathbb{F}) = 0 \rrbracket, \dots\}$. with $\mathbb{P}$ for all performance constraints and $\mathbb{R}$ for all reliability constraints, etc. For successful system integration, all COTS solutions implementing a module implementation need to satisfy the expected module properties given by M , under certain environmental conditions. Such a requirement is formally defined by module contract $\mathcal{C}$ in terms of the pair:

$$\mathcal{C} = (\varepsilon_c, \mathcal{M}_c) \quad (2)$$

with $\mathcal{M}_c$ for all COTS solutions satisfying the M ; and ε_c for all the legal environmental conditions. A contract inconsistent if $\mathcal{M}_c \neq \emptyset$ and compatible if $\varepsilon_c \neq \emptyset$. A component $M_{comp} \models C$ if and only if $M_{comp} \in \mathcal{M}_c$; An environment $E \models C$ if and only if $E \in \varepsilon_c$.

A-G (Assume-Guarantee) contract is a formalism for the description of contract C without directly referring to the actual COTS solutions $\mathcal{M}_c$. Normally, it is used for behavioral specifications of components, with A for the constraints on acceptable behaviors of the environment and G for the guarantees in terms of the corresponding component behaviors (as in e.g. [9]). Here, we use the A-G contract formalism in a more generic sense for all constraints that are characterized by logical or technical causalities from some environmental conditions (A) to some consequential module properties. Formally, an A-G contract with logical or technical causality is defined as:

$$A \Rightarrow G \quad (3)$$

with A for the constraints on the acceptable environment conditions; and G for the corresponding constraints on the properties to be guaranteed by module implementations. There is a logical or technical causality between A and G (i.e. $A \cap M \subseteq G$). Currently, we use STL (Signal temporal logic) [10] and PrSTL (Probabilistic Signal temporal logic) [14] to define the constraints.

Example for the transformation of system constraint to module contract: A vehicle braking system has been introduced in [7] as a case study for EAST-ADL. For the

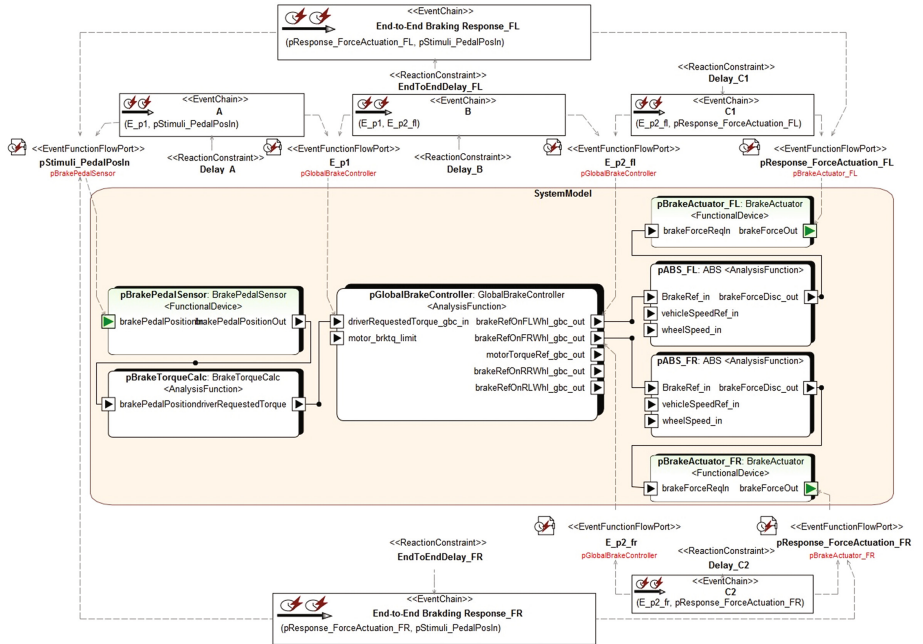


Fig. 2. The architecture model of an automotive vehicle braking system and the timing specification in EAST-ADL (implemented with the DSM Workbench MetaEdit+)

braking control, a brake controller receives the driver braking request and then sends brake force request to ABS controllers. Figure 2 shows an excerpt of the system model (*SystemModel*). The system design is converted to A-G contracts for module specification. Suppose the entire braking system will be based on COTS, the A-G contract C_{com_1} , shown in Fig. 3, stipulates the communication and timing assertions.

The timing specification in Fig. 2 augments the system model with information about the executional events and their timing constraints. In the model, an executional event defines the occurrence of data arrival on port (e.g. $pStimuli_PedalPosIn$), and the triggering of execution (e.g. E_pI). An event chain binds then these events for synchronization, e.g. to capture the end-to-end timing from sensor input to actuator outputs. In the model, the event chains *End-to-End Braking Response_FL* and *End-to-End Braking Response_FR* capture the timing requirement from driver braking request ($pStimuli_PedalPosIn$) to the brake torque actuation on the FL and FR wheels ($pResponse_ForceActuation_FL$, $pResponse_ForceActuation_FR$) respectively. For a module based implementation, the contract in Fig. 3 declares the related executional events as contract variables: x for $pStimuli_PedalPosIn$, y_1 for $pResponse_ForceActuation_FL$, y_2 for $pResponse_ForceActuation_FR$. All these events are of the types *control* and *latency critical*. Given the assumption x about its *size*, *periodicity* and *priority*, an external solution for the braking system needs not only to guarantee the preferred size and periodicity y_1 and y_2 , but also to meet the delays $x \rightarrow F_{[0.. \#delay_{y_1}]} y_1$ and $x \rightarrow F_{[0.. \#delay_{y_2}]} y_2$ for the torque response on the FL and FR wheels respectively.

$$C_{com_1}: \left\{ \begin{array}{l} \text{messages: } \begin{cases} \text{in: } x \\ \text{out: } y_1, y_2 \end{cases} \\ \text{message_types: } x, y_1, y_2 \in (Control, Latency_Critical) \\ \text{assumptions: } x = (\#size_x, \#periodicity_x, \#priority_x) \\ \text{guarantees: } y_1 = (\#size_{y_1}, \#periodicity_{y_1}), y_2 = (\#size_{y_2}, \#periodicity_{y_2}), \\ \quad G(x \rightarrow F_{[0.. \#delay_{y_1}]} y_1), G(x \rightarrow F_{[0.. \#delay_{y_2}]} y_2), \dots \end{array} \right.$$

Fig. 3. The A-G contract for the vehicle braking. Here, we use STL (Signal temporal logic) [10] to denote the constraints, with the operator G for always and F for eventually.

5 Software Services for Self-management of System Services and Components

An embedded system is self-managed if it is able to autonomously understand the actual operational situations and alter its own configurations, behaviors to meet the requirements [11]. As a further support for COTS, relating to the *Step IV*, two embedded software services have been introduced. These services, shown in Fig. 4, allow error detection and fault treatment, while complementing the verification and validation at development time with the post-deployment analysis. These two services are:

- **Monitoring and Assessment Service**, responsible for self-assessment by perceiving the operational conditions of component or system (e.g., the actual input and output conformity, the CPU and memory utilization). Contracts are used for the

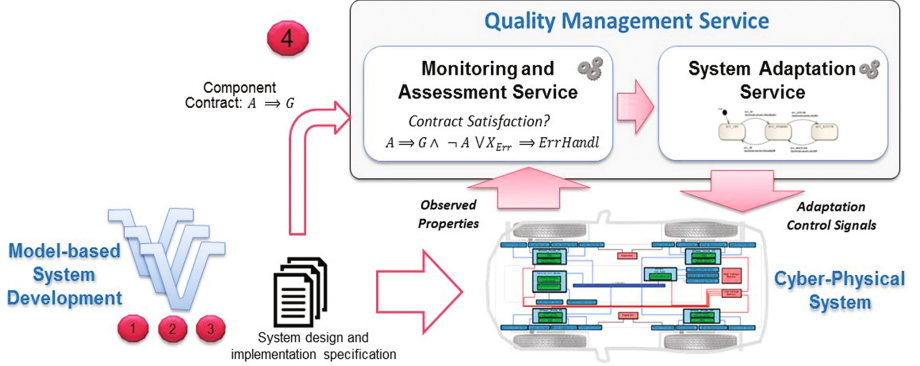


Fig. 4. The architecture of software services for self-management system services and components. (Note, ①②③④ are for the methodological steps as depicted in Fig. 1).

decision-making. For example, for a component, the service monitors the external conditions (A) and internal error states (X_{Err}); it triggers error handling ($ErrHandl$) when necessary, i.e. $A \Rightarrow G \wedge (\neg A \vee X_{Err} \Rightarrow ErrHandl)$.

- **System Adaptation Service**, responsible for planning and controlling configuration changes for quality-of-service adaptation and error handling. The decision-making requires built-in knowledge about the configuration variability. For dependability, the ability of inferring the causing factors of anomalies (e.g. based on the minimum-cut sets of fault-tree analysis) plays a key rule for the success. The adaptation is controlled by deterministic or probabilistic state-machines.

6 Related Technologies

This paper presents a methodological framework for a model-based integration and management of COTS in safety critical CPS. There are many modeling frameworks useful for the description and management of embedded systems, such as SysML [12], AADL [13] and EAST-ADL [6, 7]. All these technologies focus on the system development. For dealing with the partially unknown conditions and emergent properties, the specification with abstract goals, control policies and contracts together with the provision of intelligent services for operation supervision and adaptation becomes necessary. For example, a mission goal description based approach to the identification of required capabilities for system operations has been proposed in [15]. A contractual description of component interfaces, considering the imprecision and uncertainty, is given in [16]. For quality assurance and certification, such contractual support however needs to be managed seamlessly along with the system development and componentization. A great many techniques have recently been developed for advanced verification and validation. One promising technique is on-line model checking as explored in [17]. In this approach, the state space is continuously monitored against critical safety properties. Another approach that combines model-checking with self-assessment is

learning-based testing (LBT) [18]. New approaches to operational risk assessment also include statistical analysis of incident data using Bayesian theory [19]. In regard to all the above mentioned approaches, our work aims to facilitate the integration and adoptions of technologies.

7 Conclusion

For CPS, the need to integrate COTS solutions, supporting dependable execution and change management, calls for an integrated approach as proposed by this paper. The complexity and safety concern demand both more formal approach to system development and more advanced mechanisms for supervision and quality management. With EAST-ADL for system modeling, our work has been focused on the provision of contract formalism for module specification as well as embedded services for situation awareness and adaptation. Currently, we are looking into how to enable run-time supervision and control through the software stack of services for mixed-criticality communication based on Ethernet and AUTOSAR.

References

1. SAE International, SAE Information Report: (J3016) Taxonomy and Definitions for Terms Related to On-Road Motor Vehicle Automated Driving Systems
2. European Commission: Intelligent transport systems. https://ec.europa.eu/transport/themes/its_en
3. PwC Semiconductor Report: Spotlight on Automotive. PwC, September 2013
4. ISO, ISO 26262 Road vehicles – Functional safety
5. Chen, D., et al.: A Knowledge-in-the-loop approach to integrated safety&security for cooperative system-of-systems. In: IEEE 7th International Conference on Intelligent Computing and Information Systems, ICICIS 2015, Cairo, Egypt, 12–14 December (2015)
6. EAST-ADL. EAST-ADL Domain Model Specification, Version M.2.1.12 (2014). <http://www.east-adl.info/>
7. Kolagari, R., et al.: Model-based analysis and engineering of automotive architectures with EAST-ADL: revisited. *Int. J. Conceptual Struct. Smart Appl. (IJCSSA)* 3(2), 25–70 (2015)
8. Benveniste, A., et.al.: Multiple viewpoint contract-based specification and design. In: *Formal Methods for Components and Objects: 6th International Symposium, FMCO*, pp. 200–225 (2007)
9. Benveniste, A., et.al.: Contracts for system design. Research Report 8147, Inria, November 2012
10. Maler, O., Nickovic, D.: Monitoring temporal properties of continuous signals. In: *Joint International Conference on Formal Modelling and Analysis of Timed Systems, and Formal Techniques in Real-Time and Fault-Tolerant Systems (FORMATS/FTRTFT 2004)* (2004)
11. Anthony, R., et al.: Context-aware adaptation in DySCAS. *Electronic Communications of the EASST*, vol. 19. European Association of Software Science and Technology (EASST) (2009)
12. SysML. OMG Systems Modeling Language (OMG SysML™), OMG
13. AADL Architecture Analysis and Design Language, SEI. Carnegie-Mellon Univ., USA

14. Sadigh, D., Kapoor, A.: Safe Control under Uncertainty with Probabilistic Signal Temporal Logic. *Robotics: Science and Systems (RSS)*, June 2016
15. Silva, E., Batista, T., Oquendo, F.: A mission-oriented approach for designing system-of-systems. In: *Proceedings of the 10th System-of-Systems Engineering Conference (SoSE)*, May 2015
16. Bryans, J., Fitzgerald, J., Payne, R., Miyazawa, A., Kristensen, K.: SysML contracts for systems of systems. In: *IEEE Systems of Systems Engineering Conference*, June 2014
17. Althoff, M., et al.: Online verification of automated road vehicles using reachability analysis. *IEEE Trans. Robot.* **30**(4), 903–918 (2014)
18. Meinke, K., et al.: Incremental learning-based testing for reactive systems. In: *Proc. Int. Conf. on Tests and Proofs TAP 2011. LNCS*, vol. 6706, Springer (2011). *IEEE Trans. Robot.* **30**(4): 903–918 (2014)
19. Meel, A.: Plant-specific dynamic failure assessment using Bayesian theory. *Chem. Eng. Sci.* **61**, 7036–7056 (2006)